

Week 5 Write Up for
Master of Science
Information and Communications Technology

Melissa Knapp
University of Denver University College

April 30, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Table of Contents i

Overview 1

Challenges Faced..... 1

JUnit Testing 2

Future Considerations..... 4

Overview

The primary challenge pertains to tracking visitors across various parking facilities and invoicing them monthly according to the duration of their parking. The system encompasses functionality for registering customers and their vehicles, as well as documenting parking locations and time entries. This week, we used the Observer design pattern to notify the TransactionManager when a new ParkingTransaction occurs.

Though it wasn't part of the assignment, I did finally address one of my future considerations which was implementing the Logger class in the RegisterCarCommand and RegisterCustomerCommand classes. I spent some time learning how that worked and after implementing them, I was able to see the messages in the log when I purposefully triggered them in the JUnit tests.

Challenges Faced

I found this week to be more challenging than all the past weeks, which is partially why I implemented the Logger class, so that I felt like I was making progress. I didn't have trouble with the Observer pattern itself, but in how to link the Observer pattern to the existing architecture without changing too much, while also making sure that the transactions got logged properly and were accessible. At one point I had changed so much that I had to look back at the first week codebase to see what some of the methods originally were.

Because of this difficulty I restarted this week using last weeks codebase and started with the most basic of the classes, the ParkingEvent. Since that creation was simple, just some attributes and getters and setters, it was quick to get put together and tested. After that I moved on to the ParkingObserver class. This was straightforward as it primarily involved the

update method. Then I changed the ParkingLot class's methods to use notifyObserver(), which again was quick. Much of my time was spent with the TransactionManager trying to figure out how to get it to log a new transaction with the RealParkingOffice when a car parks.

JUnit Testing

The process of testing the updated methods was tedious as it necessitated the creation of many different objects, in a specific order, to test the methods required. I am certain that this isn't necessary and there is a less repetitive way of instantiating similar objects for each test, particularly using BeforeEach and/or BeforeAll. The main problem I ran into during testing is that whenever I tried to get parking charges the result was always a Money object of 0, when it should have been \$5.00 as I was using lot W and that is the base rate.

After a lot of trial and error it occurred to me that I needed a way to get the TransactionManager from the RealParkingOffice to get the list of transactions that I needed. After implementing the getter, I was able to retrieve the TransactionManager and get all the tests to pass. However, it was at this point I realized that the code I started with in week 1 had the TransactionManager as an attribute of the RealParkingOffice, and it had the RealParkingOffice as an attribute of the TransactionManager. While all my tests pass with this configuration, intuitively it doesn't seem like this is a good idea to have them reference each other.

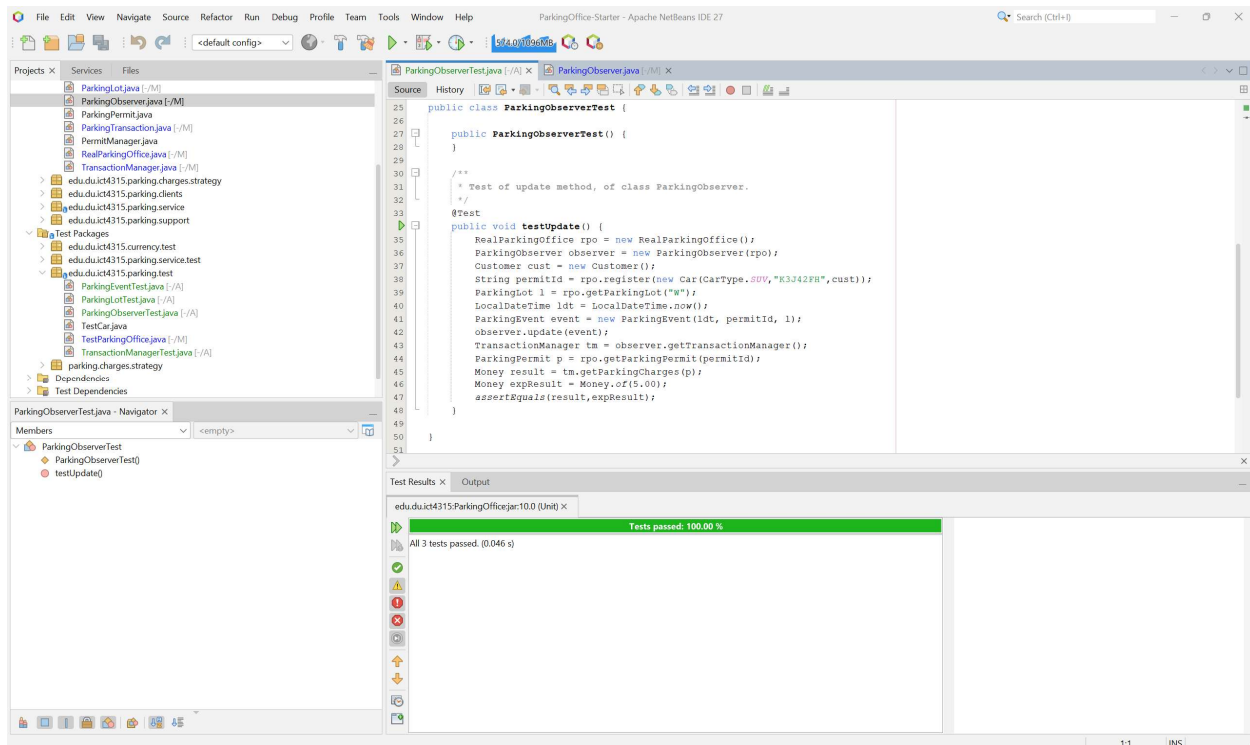


Figure 1. JUnit test results for ParkingObserverTest

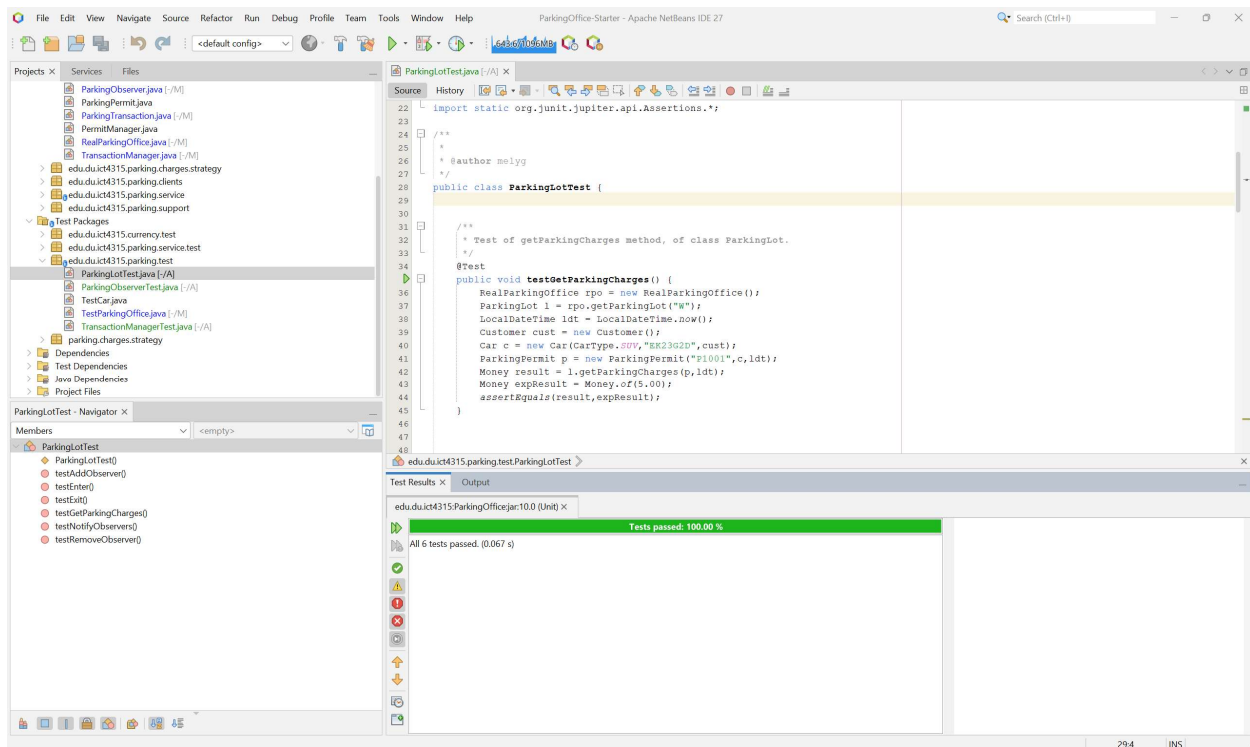


Figure 2. JUnit test results for ParkingLotTest

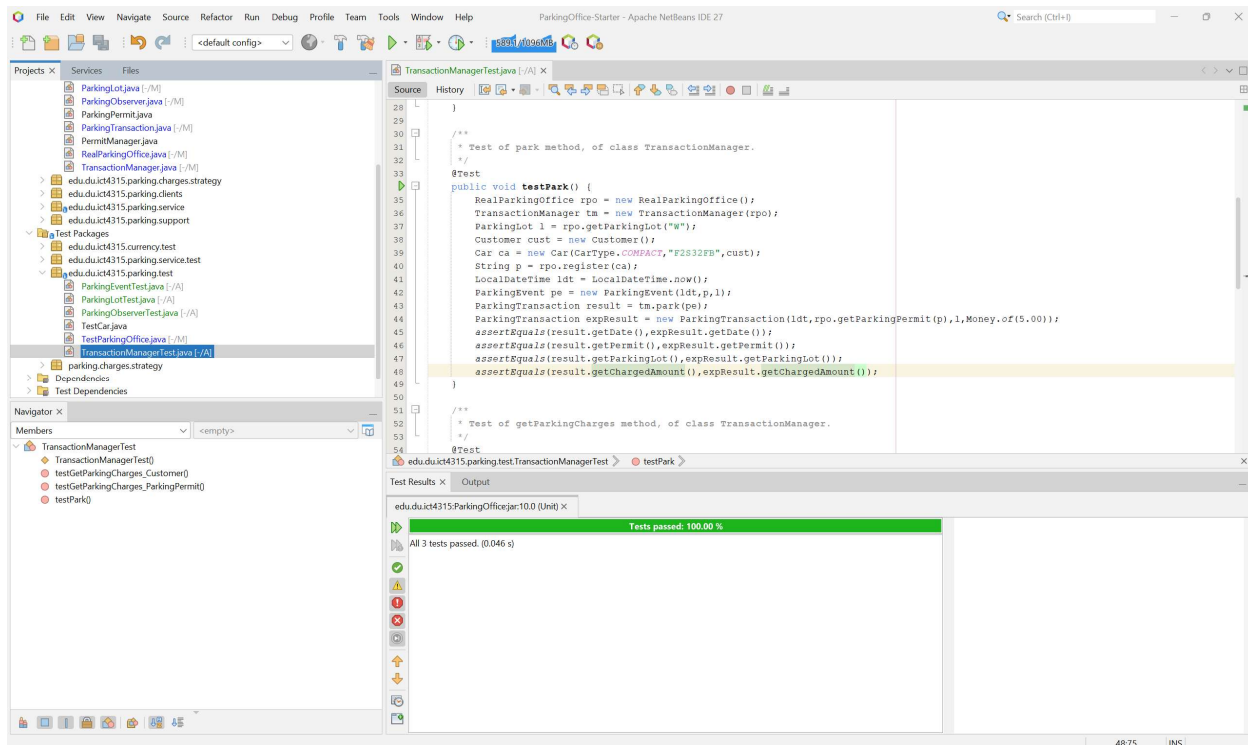


Figure 3. JUnit test results for TransactionManagerTest

Future Considerations

As I mentioned in last week's write up, I am worried that the current iteration of the parking system doesn't charge people correctly, right now it just charges the base amount no matter what parking lot is used and no matter how long the stay is. I just don't quite understand how I should be setting up the calculations and where I should be setting them up. The other concern I have is that my code is becoming too tightly coupled and I'm not sure how to address that. I might do some research on ways to de-couple code.

Additionally, I would like to gain a better understanding of the JUnit `BeforeEach` and `BeforeAll` methods. As I am currently unfamiliar with these features, I believe that learning about them could ease the tediousness of repeatedly instantiating objects for testing purposes. By exploring how `BeforeEach` and `BeforeAll` function within JUnit, I anticipate streamlining my

testing process. From what I understand, BeforeEach runs before every test, while BeforeAll executes once before all tests. Gaining proficiency with these features should make my test cases more organized and efficient, reducing repetitive code and helping ensure a consistent testing environment.